Table of contents

Abstract Interpretation

Introduction

Abstract interpretation is a formal method for analyzing the properties of computer programs. It is based on the idea of approximating program semantics in a conservative manner.

Program Correctness Problem

Several approaches exist:

- **Hoare Logic** (axiomatic semantics)
  - Deductive reasoning
  - SMT solvers (Satisfiability Modulo Theories)
- **Abstract Interpretation**

The goal is to determine whether a program satisfies certain safety properties.

The Importance of Semantics: An Example

To illustrate how the exact semantics of a language influences execution, let's compare two equivalent programs in C and Ada.

C Program:

```c
int i, a;
a = 5;
for (i = 0; i < a; i++) {
  a--;
  printf("i = %d", i);
}
```

Result: `i = 0`, `i = 1`, `i = 2`

Ada Program:

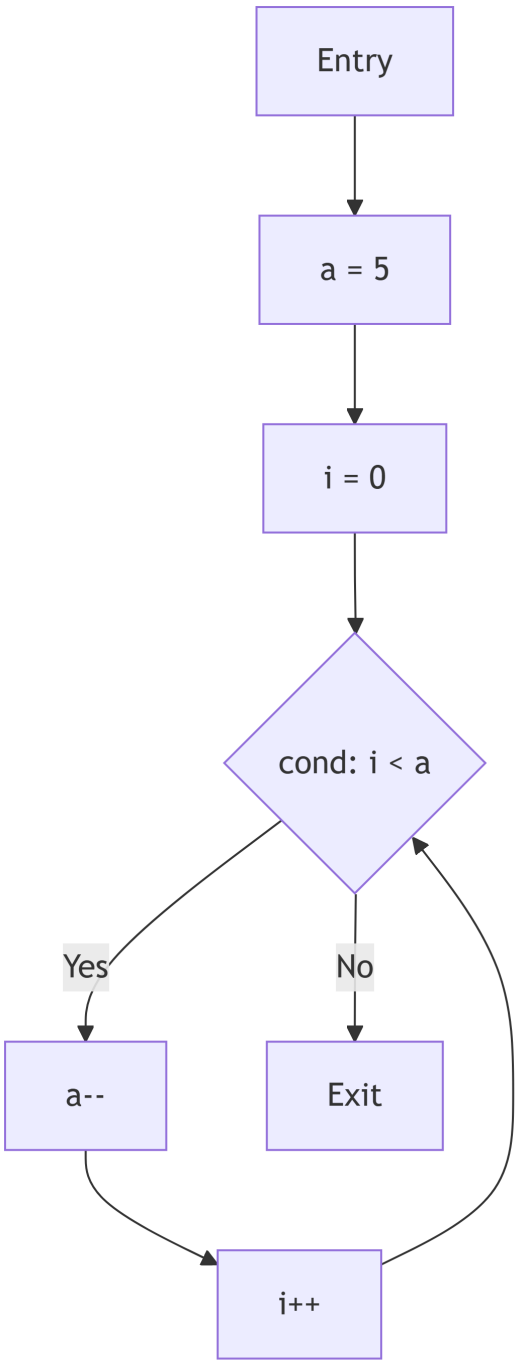
```
A: integer;
begin
  a := 5;
  for I in 0..A loop
    A := A - 1;
    Put("i="); Put(I);
  end loop;
end;
```

Result: `i = 0`, `i = 1`, `i = 2`, `i = 3`, `i = 4`, `i = 5`

Explanation: In Ada, the upper bound of the `for` loop is evaluated only once before entering the loop (copy of the value of `A`), whereas in C the condition `i < a` is reevaluated at each iteration with the current value of `a`.

Modeling with Automata

We can represent the control flow of these programs using automata. Here is the automaton corresponding to the C program:



For the Ada program, the automaton would be different because the value of `A` in the condition is fixed before the loop.

Undecidability and Approximation

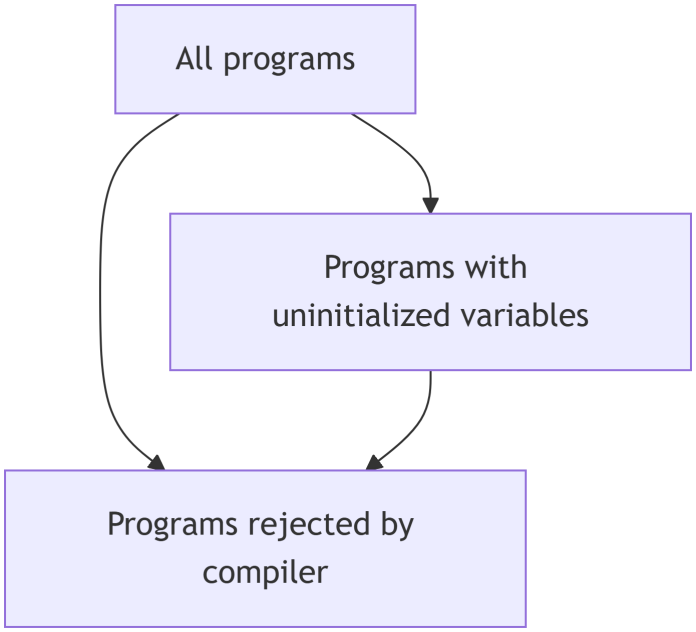
Consider the following Java example:

```
{
  int a;
  for (int i = 0; i <= a; i++) {
    // blablabla
  }
}
```

The Java compiler will refuse to compile this code or issue a serious warning: *"Variable 'a' might not have been initialized"*.

Problem: Detecting uninitialized variables is an **undecidable** (non-computable) problem.

The compiler therefore gives an **approximation** – more precisely an **over-approximation**.



The set of rejected programs is **larger** than the set of truly problematic programs. This is a **conservative approximation**:

- If the program is **not rejected**, we are sure it does **not** have an uninitialized variable problem.
- If it is rejected, we do **not know** whether it actually has a problem (false positive possible).

Collecting Semantics

General Principle

The analysis must be **conservative**: it computes a conservative approximation (abstraction) such that the answer is safe.

- If the analysis does **not reject** the program → its correctness is **guaranteed**
- If the analysis **rejects** the program → answer "maybe" (there may be an error or not)

Language Studied

The language used in this chapter is the same as in the first chapter:

- Integer and boolean expressions
- Instructions: `if`, `while`, assignments
- **Prohibition** of function calls (recursive or not)

Control Flow Graph (CFG)

Let P be a program. A **control flow graph** for P is an interpreted automaton (S, T, s_0, Var) where:

- S: finite set of states
- $s_0 \in S$: initial state
- Var: set of variables used in P
- $T \subseteq S \times (A(Var) \cup B(Var)) \times S$: set of transitions
  - $A(Var)$: set of assignments $x := expr$ where $x \in Var$ and $expr$ is an expression over Var
  - $B(Var)$: set of conditions (boolean expressions over Var)

Construction of a CFG is done recursively on each instruction of P.

Definition of Collecting Semantics

Collecting semantics (Floyd-Hoare) associates with each state (control point) of the CFG the set of all values that variables can take during execution.

Let α be a control point. We denote $V(\alpha)$ this set:

- For a variable x, $V(\alpha)(x)$ is the set of possible values that x can take at point α

Calculation Rules

1. **Assignment transition** $x := expr$ from A to B:

$$V(B) = V(A)[x := expr]$$

   where $V(B)$ takes all values from $V(A)$ except for $V(B)(x)$ which takes the values obtained by evaluating $expr$ on $V(A)$.

2. **Condition transition** $cond \in B(Var)$ from A to B:

$$V(B) = V(A) \cap [cond]$$

   where $V(B)$ takes all values from $V(A)$ that satisfy condition $cond$.

3. **Confluence** (multiple transitions to C):

   - If we have transitions from A to C and from B to C
   - First compute the effects: $V'(A)$ and $V'(B)$
   - Then: $V(C) = V'(A) \cup V'(B)$

Fixed Point Equation

Programs with loops create cycles in the CFG. Considering V as a vector $(V(A), V(B), ...)$, we obtain an equation where V depends on itself:

$$V = \Psi(V)$$

where Ψ is the function derived from the control flow graph of P.

This is a **fixed point equation**: the solution V of collecting semantics is a solution of $X = \Psi(X)$.

Fixed Point Theory

Fundamental Definitions

Partially ordered set (poset): $(X, \leq)$ where $\leq$ is a reflexive, antisymmetric, and transitive relation.

Lattice: poset $(X, \leq)$ such that for any pair $A, B \in X$:

- The infimum $\inf(A, B)$ exists in X
- The supremum $\sup(A, B)$ exists in X

Complete lattice: lattice $(X, \leq)$ such that every subset of X has an infimum and a supremum in X.

Note: Every finite non-empty lattice is complete. Only infinite lattices can be incomplete.

Upper and Lower Bounds

Let $(X, \leq)$ be a poset, $Y \subseteq X$:

- **Upper bound** of Y: $u \in X$ such that $\forall y \in Y, y \leq u$
- **Lower bound** of Y: $l \in X$ such that $\forall y \in Y, l \leq y$
- **Supremum** $\sup Y$: least upper bound of Y
- **Infimum** $\inf Y$: greatest lower bound of Y

Examples of Lattices

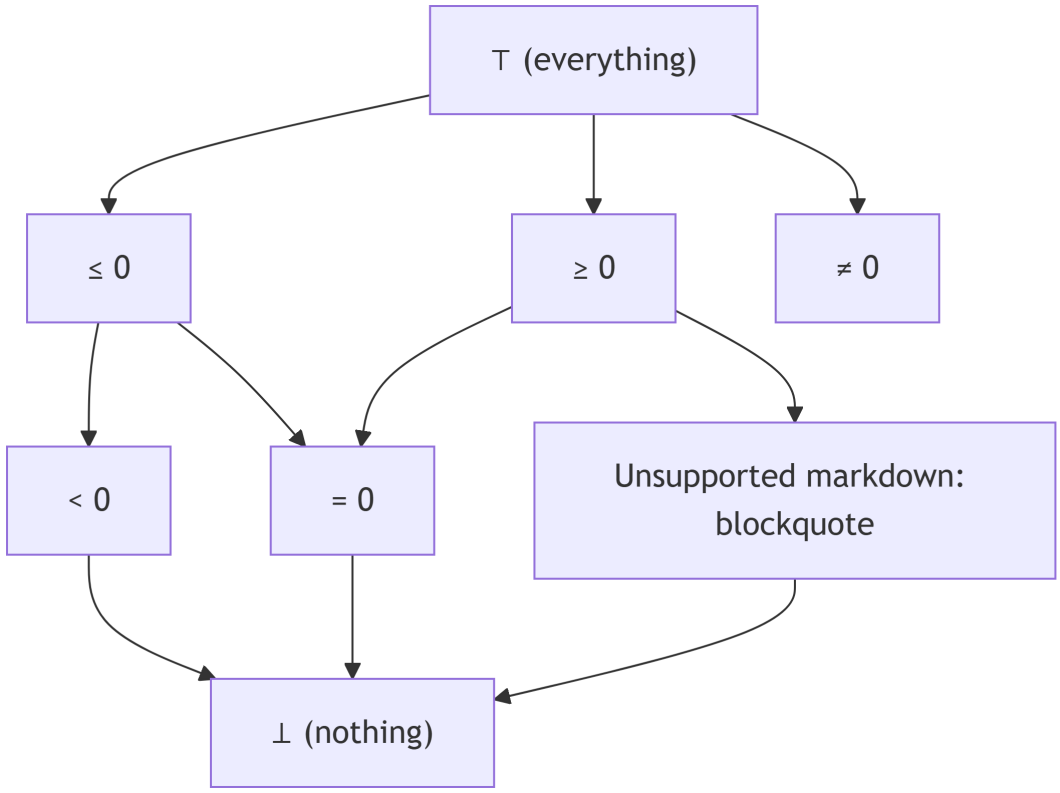
1. $(\mathbb{Z}, \leq)$: lattice but **not complete**
2. $(\mathcal{P}(\mathbb{Z}), \subseteq)$: **complete** lattice
   - $\inf(A, B) = A \cap B$
   - $\sup(A, B) = A \cup B$
   - $\sup Y = \bigcup_{S \in Y} S$
   - $\inf Y = \bigcap_{S \in Y} S$
3. **Intervals**:

   $$\mathcal{I} = \{[a, b] \mid a, b \in \mathbb{Z}, a \leq b\} \cup \{(-\infty, a] \mid a \in \mathbb{Z}\} \cup \{[a, +\infty) \mid a \in \mathbb{Z}\} \cup \{\bot, (-\infty, +\infty)\}$$

   with an order representing "precision" (see exercises): complete lattice.
4. **Signs**:

   $$\mathcal{S} = \{< 0, \leq 0, > 0, \geq 0, = 0, \neq 0, \bot, \top\}$$

   with an order representing precision: complete lattice.



Monotonic and Scott-Continuous Functions

Monotonic function: Let $(E, \leq)$ and $(F, \leq)$ be two lattices. $f : E \to F$ is **monotonic** (non-decreasing) iff:

$$\forall x, y \in E, x \leq y \Rightarrow f(x) \leq f(y)$$

Scott-continuous function: Let $(E, \leq)$ and $(F, \leq)$ be two complete lattices. $f : E \to F$ is **Scott-continuous** iff for any non-decreasing sequence $U_0 \leq U_1 \leq ... \leq U_n \leq ...$ in E:

$$\sup\{f(U_n), n \geq 0\} = f(\sup\{U_n, n \geq 0\})$$

Properties:

- If f is continuous, then f is monotonic
- Composition preserves monotonicity
- Composition preserves Scott-continuity

Fundamental Theorems

Knaster-Tarski Theorem

Let $(E, \leq)$ be a complete lattice. Let $f : E \to E$ be a monotonic function.

The set of fixed points of f, $FP = \{x \in E \mid f(x) = x\}$, is a complete lattice. In particular, f has a **least fixed-point** and a **greatest fixed-point**.

Kleene Theorem

Let $(E, \leq)$ be a complete lattice. Let $f : E \to E$ be a Scott-continuous function.

The least fixed point of f is equal to:

$$\mathrm{lfp}(f) = \sup\{f^n(\bot), n \geq 0\}$$

where $\bot = \inf E$ and f^n is defined inductively by:

- $f^0(x) = x$
- $f^{n+1}(x) = f(f^n(x))$

Property: Let $(E, \leq)$ and $(F, \leq)$ be two complete lattices. Let $f, g : E \to F$ be two continuous functions such that $\forall x \in E, f(x) \leq g(x)$. Then:

$$\mathrm{lfp}(f) \leq \mathrm{lfp}(g)$$

Fixed Point Calculation Algorithm

Kleene's theorem provides an algorithm to approximate the least fixed point:

```
X :=
repeat
  X' := f(X)
until X' = X
return X
```

Caution: If the lattice is infinite, this algorithm may not terminate.

Product of Lattices

Lemma: Let $(E, \leq)$ and $(F, \leq)$ be two complete lattices. We define the order on $E \times F$ by:

$$\forall (a, b), (c, d) \in E \times F, (a, b) \leq (c, d) \text{ iff } a \leq c \text{ and } b \leq d$$

Then $(E \times F, \leq)$ is a complete lattice.

Application: For a program with $\#v$ variables and $\#cp$ control points, the equation $X = \Psi(X)$ is expressed on the lattice $\mathcal{I}^{\#v \times \#cp}$ (if we use intervals).

Galois Connection

Safety and Liveness Properties

Program specifications can be divided into two types:

- **Safety**: "Nothing bad will ever happen"
  - Violated in finite time
  - Example: no division by zero
- **Liveness**: "Something good will eventually happen"
  - Example: termination

We focus on **safety properties**.

Safety Observer

A safety property can be expressed using an **observer** (automaton) that:

- Reads program values
- Moves to an ERROR state when the property is violated

We can transform the CFG so that the safety property is satisfied iff the ERROR state is unreachable.

Undecidability Problem

Reachability of a control point is **not decidable** in general. We therefore use collecting semantics on an abstract domain to obtain a conservative answer.

Conservative analysis:

- Answer "OK" (no error) → **guarantee** of correctness
- Answer "I don't know" → possible error (or not)

Analysis Method

1. **Choose an abstract domain** (signs, intervals, …)
2. **Express the safety property as an observer**
3. **Transform the program and observer** into a CFG with ERROR state
4. **Express as fixed point equations**
5. **Calculate (try) the least fixed point**
    - If the ERROR state is reached (value $\neq \bot$) → "I don't know"
    - If a fixed point is reached without ERROR → "OK"
    - (This step may not terminate)

Definition of Galois Connection

A **Galois connection** between:

- A concrete complete lattice $(E, \leq)$
- An abstract complete lattice $(F, \preceq)$

is a pair of functions $\alpha : E \to F$ (abstraction) and $\gamma : F \to E$ (concretization) such that:

$$\forall x \in E, \forall y \in F, \alpha(x) \preceq y \iff x \leq \gamma(y)$$

Properties

1. $\gamma \circ \alpha$ is **extensive**: $\forall x \in E, x \leq \gamma(\alpha(x))$
2. $\alpha \circ \gamma$ is **intensive**: $\forall y \in F, \alpha(\gamma(y)) \preceq y$
3. α and γ are monotonic
4. α is Scott-continuous

Application to Abstract Interpretation

In our context:

- **Concrete lattice**: $\mathbb{Z}^{\#v}$ (values of variables)
- **Abstract lattice**: $\mathcal{I}^{\#v}$ (intervals)
- **Abstraction** α: transforms sets of values into intervals
- **Concretization** γ: transforms intervals into sets of values

Collecting semantics is expressed on the concrete side: $V = \Psi(V)$

Fixed point calculation is done on the abstract side: $W = \Psi_{\text{abst}}(W)$

We can take:

$$\Psi_{\text{abst}} = \alpha \circ \Psi \circ \gamma$$

Provided that Ψ and γ are continuous.

Correctness of Abstract Analysis

By calculating $\text{lfp}(\Psi_{\text{abst}})$, we obtain an **over-approximation**, hence a conservative answer. We can prove that:

$$\gamma(\text{lfp}(\Psi_{\text{abst}})) \geq \text{lfp}(\Psi)$$

Sometimes, we use an even more abstract function ξ such that $\forall y \in F, \Psi_{\text{abst}}(y) \leq \xi(y)$ to accelerate calculation and ensure termination. In this case too, we obtain an over-approximation:

$$\gamma(\text{lfp}(\xi)) \geq \text{lfp}(\Psi)$$

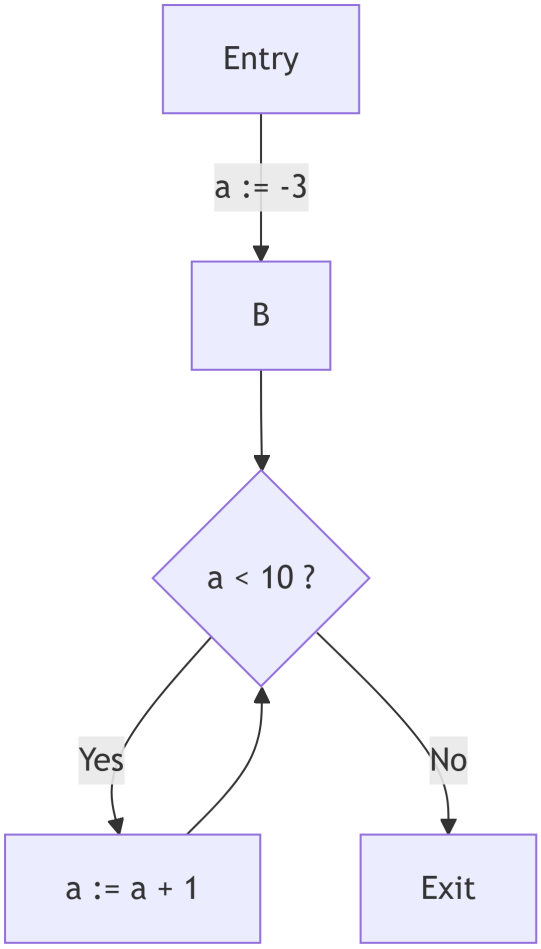
Complete Analysis Example

Example Program

Consider the simple program:

```
a = -3
while a < 10 do
  a = a + 1
done
```

CFG Construction



Collecting Semantics Equations

On the signs domain $\mathcal{S}$:

- $V(A) = \top$
- $V(B) = V(A)[a := -3] \cup V(D)[a := a + 1]$
- $V(C) = V(B) \cap [a < 10]$
- $V(D) = V(B) \cap [a \geq 10]$

We look for $V = \phi(V)$ where $V \in \mathcal{S}^4$.

Iterative Calculation

1. Initialization: $V^0 = (\bot, \bot, \bot, \bot)$
2. Iteration 1: $V^1 = \phi(V^0)$
   - $V^1(A) = \top$
   - $V^1(B) = \top[a := -3] \cup \bot[a := a + 1] = \{a = -3\} \cup \bot = \{a = -3\}$
   - $V^1(C) = \{a = -3\} \cap [a < 10] = \{a = -3\}$
   - $V^1(D) = \{a = -3\} \cap [a \geq 10] = \bot$
3. Iteration 2: $V^2 = \phi(V^1)$
   - Continue until convergence...

Analysis with Safety Property

Suppose we want to verify that a never becomes positive. We add an observer that moves to ERROR if $a > 0$.

The abstract analysis will determine whether the ERROR state is reachable.

Comparison of Verification Methods

Method	System	Property	Technique
Hoare Logic	Program	Pre/post-conditions	Deductive proofs
SMT Solving	Logical formula	Formula to verify	Automatic solvers
Abstract Interpretation	Program	Safety property (observer)	Fixed point on abstract domain
Model Checking	Finite automaton	Temporal logic (LTL, CTL)	State exploration

For abstract interpretation, we verify that the set of system traces is included in the set of property traces:

$$\text{Tr(System)} \subseteq \text{Tr(Property)}$$

Conclusion

Abstract interpretation is a powerful method for static analysis of programs. It allows verifying safety properties in a conservative manner, with formal guarantees thanks to the theory of complete lattices and Galois connections.

Key points to remember:

1. The analysis is **conservative** (safe but not complete)
2. It relies on **collecting semantics** and **fixed point equations**
3. **Lattice theory** and **Galois connections** provide the mathematical foundation
4. The choice of **abstract domain** influences the precision and efficiency of the analysis